

CENG 232

Logic Design

Spring 2025-2026

Lab Assignment 3

Due date: Monday, May 18, 2026, 23:59

Introduction

This laboratory aims to familiarize you with some of the IC components that are crucial for a sequential logic circuit, such as a D-Flip Flop, J-K Flip Flop. You are expected to design your circuit using the CENG232 Logisim tool with the given gates below and implement it via Verilog HDL. Unlike the previous assignments, a report to answer questions in the sections should be prepared in LaTeX.

In combinational circuits, outputs depend only on the current inputs. In sequential circuits, outputs depend on both current inputs and previously stored state. That stored state is maintained by **storage elements**, most fundamentally **latches** and **flip-flops**. A **latch** is **level-triggered**: while its enable signal is active, the output continues to follow the input. A **flip-flop** is **edge-triggered**: it samples the input only at a specific clock transition and holds that value until the next active edge.

In this assignment, you are expected to design storage elements from the gate level upward. You need to first construct a D latch from either NAND gates or NOR gates, then cascade two latches to form a **master-slave, positive-edge-triggered D flip-flop**. Next, you need to extend your flip-flop with an **asynchronous active-low preset and active-high clear**, and then use that custom D flip-flop to realize a **JK flip-flop**.

IC Pool

- 74HC08 (AND)
- 74HC32 (OR)
- 74HC04 (NOT, Inverter)
- 74HC02 (NOR)
- 74HC00 (NAND)
- 74HC153 (Multiplexer)
- 74HC86 (XOR)
- 74HC138 (Decoder)
- 74HC74 (D-Flip Flop)

Naming conventions used in this assignment:

Signal Name	Meaning	Active level
D	data input	active-high logic level
J, K	JK control inputs	active-high logic level
EN	latch enable	active when 1
CLK	system clock	capture on rising edge
Q	main output	—
$\overline{Q}$	complementary output	—
$\overline{PRE}$	asynchronous preset	active-low logic level
CLR	asynchronous clear	active-high logic level

Part 1: NAND/NOR-Based D Latch Design

A D latch has a data input D , and an enable input EN. When $EN = 1$, the latch is **active**: changes at D propagate to Q . When $EN = 0$, the latch is in **hold** mode and retains its previous state.

Expected behavior:

Table 1: Expected D-latch behavior

EN	D	Q_{next}	$\overline{Q}_{\text{next}}$	Mode
0	X	Q_{prev}	$\overline{Q}_{\text{prev}}$	Hold
1	0	0	1	Active
1	1	1	0	Active

- While $EN = 1$, the latch is open and Q may keep changing if D keeps changing.
- While $EN = 0$, the latch should preserve its previously stored value even if D changes.

Required module interface for D-Latch:

```
1 module d_latch(  
2     input D,  
3     input EN,  
4     output Q,  
5     output Q_bar  
6 );  
7     // Gate-level only (nand, nor, not), no reg variables are allowed  
8     // Allowed primitives inside this module: nand, nor, not  
9 endmodule
```

D-Latch Related Questions to answer in your report:

1. Why is the D latch called **level-triggered**? Please use the behavior of EN in your explanation.
2. While $EN = 1$, does the output change at exactly the same instant as the input? If not, what physical effect causes the difference?

3. What is the forbidden or contradictory condition of the underlying SR core, and how does your D-latch design prevent it?
4. If D changes just before the latch closes, what value do you expect the latch to store? Please explain using the timing of the enable transition.

Part 2: Master–Slave Positive-Edge-Triggered D Flip-Flop

A single D latch is active during its enable window. To obtain edge-triggered behavior, two latches can be cascaded in a **master-slave** arrangement. The **master** latch is active when $\text{CLK} = 0$ and therefore uses $\overline{\text{CLK}}$ as its enable. Similarly, the **slave** latch is active when $\text{CLK} = 1$ and therefore uses CLK as its enable.

At the rising edge of the clock, the master stops tracking the input, and the slave becomes active. The slave, therefore, receives the value already stored by the master just before the edge.

Expected behavior:

Table 2: Expected master-slave D-flip-flop behavior. 'X' stands for 'don't care'

Clock condition / transition	D	Q_{next}	$\overline{Q_{\text{next}}}$	Mode
Stable 0	X	Q_{prev}	$\overline{Q_{\text{prev}}}$	Hold
Stable 1	X	Q_{prev}	$\overline{Q_{\text{prev}}}$	Hold
Rising edge $\uparrow$	0	0	1	Capture
Rising edge $\uparrow$	1	1	0	Capture
Falling edge $\downarrow$	X	Q_{prev}	$\overline{Q_{\text{prev}}}$	No output update

- Changes at D while $\text{CLK} = 0$ may affect the internal master node but should not directly affect the external output.
- Changes at D while $\text{CLK} = 1$ should not pass through to the output because the master is closed.

0.1 Required module interface for positive-edge-triggered D Flip Flop:

```

1 module d_ff(
2     input D,
3     input CLK,
4     output Q,
5     output Q_bar
6 );
7     // Gate-level only (and, or, not, etc.), no reg variables are allowed
8     // Please build by instantiating two d_latch modules and one inverter
9 endmodule

```

Positive-Edge-Triggered D-Flip Flop Related Questions to answer in your report:

1. Please explain, using one concrete input scenario, the difference between the level-triggered D latch of Part 1 and the edge-triggered D flip-flop of Part 2.
2. Why must the master and slave not be active at the same time? Please explain race-through in your own words.
3. If your manual clock source bounces, how can that distort the expected behavior? What symptoms would you expect at the output?

Part 3: D Flip-Flop with Asynchronous Active-Low Preset and Asynchronous Active-High Clear

Many practical storage elements include asynchronous control inputs that override the clock-driven data information. For this part, you are expected to add **asynchronous, active-low preset** and **asynchronous, active-low clear** inputs named $\overline{PRE}$ and CLR to your Part 2 D flip-flop. If $\overline{PRE}=CLR=0$, the output must be forced to $Q = 1$ and $\overline{Q} = 0$ without waiting for a clock edge. If $\overline{PRE} = 1, CLR = 1$, the output must be forced to $Q = 0$ and $\overline{Q} = 1$ without waiting for a clock edge. When $\overline{PRE} = 1, CLR = 0$ the flip-flop must behave exactly like the Part 2 positive-edge-triggered D flip-flop.

Expected behavior:

Table 3: Expected behavior of (positive-edge-triggered) D-Flip Flop with asynchronous preset and reset

$\overline{PRE}$	CLR	Clock condition / transition	D	Q_{next}	$\overline{Q}_{\text{next}}$	Mode
0	0	X	X	1	0	Asynchronous preset
1	1	X	X	0	1	Asynchronous clear
0	1	X	X	0	1	Asynchronous clear
1	0	Stable 0 or 1	X	Q_{prev}	$\overline{Q}_{\text{prev}}$	Hold
1	0	Rising edge $\uparrow$	0	0	1	Capture
1	0	Rising edge $\uparrow$	1	1	0	Capture
1	0	Falling edge $\downarrow$	X	Q_{prev}	$\overline{Q}_{\text{prev}}$	No output update

- Preset and clear must take effect without waiting for the clock.

0.2 Required module interface for D-Flip Flop with preset and reset:

```

1 module d_ff_psr(
2     input D,
3     input CLK,
4     input PRE_bar,
5     input CLR,
6     output Q,
7     output Q_bar
8 );
9     // reg variables are allowed
10
11 endmodule

```

D-Flip Flop with preset Related Questions to answer in your report:

1. What does **asynchronous preset** mean? How would the behavior differ if the preset were synchronous?
2. If $\overline{PRE}$ is asserted (and CLR is asserted 0) while $D = 0$ and the clock is switching, why must the output still go to 1?
3. Please describe a scenario where the preset input is handy for a D-Flip Flop.
4. Please describe a scenario where the clear input is handy for a D-Flip Flop.

Part 4: JK Flip-Flop Design Using D-Flip-Flop

In this part, you are expected to implement a JK-Flip Flop using a Part 3 D-Flip Flop and logic gates. A J-K flip-flop provides hold, reset, set, and toggle behavior.

Expected behavior:

Table 4: JK-Flip Flop behavior

J	K	Q_{next}	$\overline{Q_{\text{next}}}$	Mode
0	0	Q_{prev}	$\overline{Q_{\text{prev}}}$	Hold
0	1	0	1	Reset
1	0	1	0	Set
1	1	$\overline{Q_{\text{prev}}}$	Q_{prev}	Toggle

- When $J = 1, K = 0$, the output (Q) should be 1 per rising clock edge.
- When $J = 0, K = 1$, the output (Q) should be 0 per rising clock edge.
- When $J = K = 1$, the output should toggle once per rising clock edge.
- As Part 3 D-Flip Flop is utilized, the J-K Flip Flop should already support preset and clear functionalities.

Required module interface for J-K Flip-Flop:

```
1 module jk_ff(  
2     input J,  
3     input K,  
4     input CLK,  
5     input PRE_bar,  
6     input CLR,  
7     output Q,  
8     output Q_bar  
9 );  
10     // Gate-level only (and, or, not, etc.), no reg variables are allowed  
11     // Please use the d_ff_psr module and logic gates  
12 endmodule
```

J-K Flip-Flop Related Questions to answer in your report:

1. Please write the behavior equation of the J-K Flip-Flop, including the D-Flip Flop inputs and outputs.
2. How many additional gates did Part 4 require beyond Part 3?
3. Why is the JK front-end logically richer than simply wiring D directly, and what extra timing sensitivity can that introduce in practice?

Specifications

- For Logisim design, each part should be implemented in a separate file: part1.circ, part2.circ, part3.circ, and part4.circ.
- For Verilog, you're expected to implement all four modules (d_latch, d_ff, d_ff_psr, jk_ff) in a single file: eXXXXXXX.v (Xs should be replaced with your student ID numbers).

- For your answers to the questions, you are expected to prepare a single report file (PDF) prepared in LaTeX.
- In the `d_latch` module, you're expected to use only gate-level primitives provided by Verilog (e.g., `nor`, `nand`, `not`). You are not allowed to use a reg variable to store information.
- In the `d_ff` module, you're expected to use only gate-level primitives provided by Verilog (e.g., `not`, `and`, etc.) and instances of your `d_latch` module implementation. You are not allowed to use a reg variable to store information.
- In the `d_ff_psr` module, you can utilize reg variables and other gate-level primitives provided by Verilog (e.g., `not`, `and`, etc.), along with instances of your `d_latch` module implementation.
- In the `jk_ff` module, you're expected to use only gate-level primitives provided by Verilog (e.g., `not`, `and`, etc.) and instances of your `d_ff_psr` module implementation. You are not allowed to use a reg variable to store information.
- In your module implementations, you may use continuous assignments (`assign`) or `always` blocks.
- For your Part 3 Logisim design, you are not allowed to use the 74HC74 (7474) component.
- You must not use any behavioral delay statements in your module implementations.
- The Verilog implementations must produce the same outputs as your Logisim circuits for all possible input combinations.
- The evaluation will be done using automated test cases (black-box testing).

Logisim Labelling Specifications

- You have to use **pins** for your inputs and outputs. Please only set **label property** of the **pin** objects; please do not add a **text object** instead of labels.
- For Part 1, your input and output pins should be labeled as they are described in the section: EN, D, Q, Q_bar (identical to how they are defined in the module definition).
- For Part 2, your input and output pins should be labeled as they are described in the section: D, CLK, Q, Q_bar.
- For Part 3, your input and output pins should be labeled as they are described in the section: D, CLK, PRE_bar, CLR, Q, Q_bar.
- For Part 4, your input and output pins should be labeled as they are described in the section: J, K, CLK, PRE_bar, CLR, Q, Q_bar.
- Label properties are case-sensitive. **Please be very careful with the naming of labels.**
- If you need to feed any input with a constant value, you can use a constant gate. This gate is under the CENG232 Gates section. We will only set values for the input pins specified above. If you add an input pin instead of a constant gate in order to provide a 0 or 1 value for a logic gate in your design, during evaluation, your design may get very low scores even if it has been implemented correctly.
- You will receive a grade **penalty** unless labeling is done correctly.

Deliverables

1. Please submit your circuit design files named `part1.circ`, `part2.circ`, `part3.circ`, and `part4.circ` prepared in Logisim, your Verilog implementation file named `e1234567.v`, and `e1234567.pdf`, which is your preliminary work, until the specified deadline. For Verilog and report files, please do not forget to replace `e1234567` with your 7-digit student ID. The evaluation of the Logisim and Verilog submissions will be a black-box test. **You should use the CENG version of Logisim is available on the ODTUClass course page. Circuits designed with other Logisim versions, other tools, or not named properly will not be graded!**
2. In the demo session, you will reconstruct and show that the circuit drawn in Logisim works physically. This part will be graded in the lab.

Note: Your overall Lab 3 grade is going to be calculated from the Logisim assignment score, Verilog assignment score, report score, lab demo score, and quiz score.

Free Session

After your homework is announced, there will be a *free session week*. You will have 2 hours in your free session slot. At the very beginning, a couple of the new ICs (for this assignment) will be introduced. In the remaining time of the free session, you may build your solution circuit on a breadboard by using IC components and practice how to handle possible problems related to a physical circuit on a breadboard.

Demo Session

There will be a 2-hour-long *demo session week* following the free session week. In the demo session:

- You will take a short quiz about the logic concepts that involve the coverage of this lab.
- You will reconstruct your circuit on your breadboard.
- You will show that the circuit drawn in Logisim works as specified.
- A sample solution that can be implemented will be published before the first demo session (**you are not going to be asked to implement your whole design**).

What to Bring to the Demo Lab

- Circuit designs prepared with CENG232 Logisim (you can have them printed or downloaded on your phone/tablet, etc.).
- Data sheets of chips (you can have them printed or downloaded on your phone/tablet, etc.). www.alldatasheet.com
- Pencil and eraser, as you will have a quiz at the very beginning of demo sessions.

Note: The lab materials (gates, cables, etc.) will be distributed at the beginning of each lab session and will be collected at the end of the session. Therefore, you do not have to buy them. If you want to work out of the labs, you may still buy them. Moreover, if you would like to work with your own equipment (ICs, breadboard, cables, etc), you are free to bring them to the sessions as well.

Cheating Policy

All lab work should be done individually, and there is a zero-tolerance policy for cheating. Please see the course website for further information about the cheating policy.

References

- [CENG 232 Logisim](#)
- [Discussions Page](#)